CS6530 Applied Cryptography — Course Project Final Report

Post-Quantum Secure Messaging: Formal Analysis, Failure Reproduction, and Hardened Protocol Design in Signal PQXDH

Roll Numbers & Authors:

Akhilesh K.R (CS26E001)

D R Santhosh (CS26E004)

Mysha Ali (CS26E008)

Course: CS6530 Applied Cryptography (Jul–Nov 2026)

Instructor: Prof. Manikantan

Theme: Theme 2 — Post-Quantum Secure Messaging

Deliverable: Artefact 1 (Technical Report)

Abstract

The impending emergence of Cryptographically Relevant Quantum Computers (CRQCs) threatens classical public-key cryptography through Shor's algorithm, enabling adversaries to execute Harvest-Now-Decrypt-Later (HNDL) attacks against archived end-to-end encrypted messaging transcripts. In response, modern secure messaging protocols are transitioning from classical Diffie–Hellman (X3DH) to post-quantum hybrid key encapsulation mechanisms (PQXDH). However, post-quantum migration is fundamentally a protocol composition and key-binding challenge, rather than a mere primitive substitution.

In this work, we conduct an end-to-end cryptanalytic and formal verification study of the Signal PQXDH protocol based on primary literature by Bhargavan et al. (USENIX Security '24) and supporting literature [R2]–[R4]. First, we establish the formal system and Dolev-Yao threat model for hybrid key establishment (Activity 1). Second, we dissect two critical security failure modes: the KEM Re-encapsulation vulnerability arising from the gap between generic IND-CCA2 KEM security and public-key binding, and key-type confusion attacks (Activity 2). Third, we conduct a bounded formal reproduction in ProVerif 2.05, demonstrating the automated extraction of an active re-encapsulation attack trace against PQXDH Revision 1 and formally proving that Signal Revision 2 restores injective agreement and session independence via Associated Data (AD) binding (Activity 3). Fourth, we develop two novel engineering explorations: formally modeling the catastrophic impact of key-type confusion when classical and KEM public-key encodings are un-prefixed, and formally designing and proving an alternative transcript-binding architecture (Proposal P2) where KEM keys are bound directly into the Key Derivation Function (HKDF) (Activity 4). Finally, we synthesize our comparative

findings across four recent post-quantum papers into a Security Evolution Matrix (Activity 5) and distill five core engineering lessons for robust post-quantum migration (Activity 6).

---

1. Problem Statement and Primary Paper Understanding

End-to-end encrypted messaging systems such as Signal and WhatsApp have traditionally protected billions of users using the Extended Triple Diffie–Hellman (X3DH) protocol over Curve25519. In X3DH, asynchronous session establishment relies on Diffie–Hellman prekeys published to an untrusted server. While computationally secure against classical polynomial-time adversaries, X3DH provides zero forward secrecy against an adversary who records encrypted ciphertexts today and decrypts them retroactively once a quantum computer capable of calculating discrete logarithms becomes operational.

To counteract HNDL attacks, Signal proposed **PQXDH**, a hybrid key agreement protocol that combines four classical Curve25519 Diffie–Hellman operations (DH_1, DH_2, DH_3, DH_4) with an IND-CCA2-secure lattice-based Key Encapsulation Mechanism: **ML-KEM-1024 (Kyber-1024)**. The design philosophy of PQXDH was twofold:

1. Retain the classical security guarantees of X3DH even if the underlying lattice mathematics of Kyber were to be broken; and

2. Ensure quantum resistance against passive network eavesdroppers who harvest ciphertexts.

However, as revealed by Bhargavan et al. [1] in the primary paper (*"Formal Verification of the PQXDH Post-Quantum Key Agreement Protocol for End-to-End Secure Messaging"*, USENIX Security '24), integrating a post-quantum KEM into an established DH ratcheting architecture is fraught with subtle protocol-level failure modes that classical cryptanalysis and functional unit tests fail to detect. In particular, the primary paper identified three critical specification weaknesses in Signal's initial PQXDH Revision 1:

1. **The Gap in Generic IND-CCA2 Security (KEM Re-encapsulation):** IND-CCA2 guarantees that an adversary cannot invert a ciphertext under a given public key, but does *not* prevent an adversary with access to one private key from crafting an alternate ciphertext that decrypts to the *identical* shared secret under a *different* public key. In PQXDH Revision 1, this allowed an active attacker to break session independence.

2. **Public-Key Encoding Confusion:** The lack of strict disjoint typing between Curve25519 public keys and Kyber public keys created ambiguities during message serialization.

3. **One-Time vs. Last-Resort Prekey Ambiguity:** If the server served a shared "last-resort" KEM key rather than a one-time KEM prekey, an initiator had no cryptographic mechanism to verify whether the responder would immediately zeroize the private key, compromising forward secrecy.

Signal subsequently issued Revision 2 to address these vulnerabilities. Our project executes a rigorous formal analysis and bounded automated verification of these phenomena.

---

2. System and Security Model (Activity 1)

2.1 Protocol Workflow and Mathematical Specification

The Signal PQXDH protocol operates asynchronously between two users, Alice (initiator) and Bob (responder), coordinated via an untrusted Signal messaging server.

```
[ Alice (Initiator) ]                      [ Untrusted Server ]                      [ Bob
(Responder) ]
        |                                          |
|
        |                                          |<==== Publishes Prekey Bundle:
=========|
        |                                          |      IK_B, SPK_B, Sig_B, PQPK_B,
OPK_B  |
        |                                          |
|
        | 1. Request Prekey Bundle for Bob         |
|
        |----------------------------------------->|
|
        |                                          |
|
        | 2. Returns Prekey Bundle                 |
|
        |<-----------------------------------------|
|
        |                                          |
|
  Verify Sig_B(SPK_B)                              |
|
  Gen Ephemeral: EK_A                              |
|
  KEM Encaps: (CT_PQ, SS_PQ) = Encaps(PQPK_B)      |
|
  Compute DHs:                                     |
|
    DH1 = X25519(IK_A_priv, SPK_B)                 |
|
    DH2 = X25519(EK_A_priv, IK_B)                  |
|
    DH3 = X25519(EK_A_priv, SPK_B)                 |
|
    DH4 = X25519(EK_A_priv, OPK_B) [if OPK present] |
|
  SK = KDF(KMAC256(F || DH1 || DH2 || DH3 || DH4 || SS_PQ))
```

```
|                                                
```

4. **Initial Message Encryption (AEAD):** Alice encrypts the payload using authenticated encryption (AES-256-GCM):

$$\text{Ciphertext} = \text{AEAD_Encrypt}(SK, \text{Payload}, \text{AD})$$

In **Revision 1:** $\text{AD} = IK_A \parallel IK_B$

In **Revision 2:** $\text{AD} = IK_A \parallel IK_B \parallel PQPK_B$

2.2 Formal Threat Model

Attacker Capability (Dolev–Yao): The adversary has total control of the communications network. The attacker can intercept, drop, modify, replay, inject, and reorder all messages between Alice, Bob, and the server.

Compromise Model:

Ephemeral DH keys (EK_A) and one-time prekeys (OPK_B) may be compromised dynamically.

Old post-quantum prekeys ($PQPK$) may be compromised by the attacker to model state leakage or key expiration.

The server is untrusted (honest-but-curious or actively colluding with the adversary).

Quantum Adversary (HNDL): The adversary records all network ciphertexts and possesses a CRQC in the future capable of computing discrete logs on Curve25519, but cannot invert the post-quantum lattice problem (Module-LWE in Kyber).

Security Goals:

1. *Entity Authentication & Injective Agreement:* Bob must accept a session with Alice if and only if Alice initiated a session with Bob, agreeing on all parameters ($IK_A, IK_B, OPK, SPK, PQPK$).

2. *Post-Quantum Secrecy:* Even with a CRQC breaking all DH components ($DH_1 \dots DH_4$), session key SK must remain secret as long as the KEM shared secret SS_{PQ} is intact.

3. *Session Independence:* Compromise of a private key $PQSK_2$ belonging to one prekey must never allow an attacker to decrypt a session negotiated under an uncompromised prekey $PQPK_1$.

---

3. Security Failure Analysis (Activity 2)

3.1 Failure Mode 1: KEM Re-encapsulation

Theoretical Vulnerability: Most cryptographers assume that using an IND-CCA2 secure KEM is sufficient for authenticated key exchange. However, IND-CCA2 security is defined for a single, fixed public key:

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA2}}(\mathcal{A}) = \left| \Pr[\mathcal{A}(pk, c^*) = 1] - \Pr[\mathcal{A}(pk, c_{\text{rand}}) = 1] \right| \le \epsilon$$

Crucially, IND-CCA2 **does not guarantee public-key binding**. That is, given two distinct public keys PK_1 and PK_2, it may be computationally easy for an adversary who controls SK_2 to compute a ciphertext CT' such that:

$$\text{Decaps}(SK_2, CT') = \text{Decaps}(SK_1, CT)$$

Attack Walkthrough against PQXDH Revision 1:

1. Bob publishes two post-quantum prekeys: an active honest prekey $PQPK_1$, and a compromised or stale prekey $PQPK_2$ (for which the attacker knows SK_2).

2. Alice initiates a session meant for $PQPK_2$. She encapsulates CT_2 with shared secret $SS = \text{Encaps}(PQPK_2)$ and encrypts her message with $\text{AD} = IK_A \parallel IK_B$.

3. The active attacker intercepts Alice's message, decapsulates CT_2 using SK_2 to recover SS, and re-encapsulates the *identical* shared secret SS into a new ciphertext CT_1 under Bob's uncompromised key $PQPK_1$.

4. The attacker delivers $(IK_A, EK_A, CT_1, \text{Ciphertext})$ to Bob.

5. Bob decapsulates CT_1 with SK_1, deriving the identical SS. Bob derives the exact same session key SK and successfully verifies the AEAD tag because $\text{AD} = IK_A \parallel IK_B$ does not mention $PQPK$.

6. **Consequence:** Bob believes he has established a fresh, secure session with Alice under his uncompromised prekey $PQPK_1$. In reality, the session key is known to the attacker, completely violating injective agreement and session independence.

3.2 Failure Mode 2: Key-Type Confusion

Theoretical Vulnerability: When a protocol handles multiple cryptographic primitives (e.g., elliptic curve points for X25519 and polynomial matrices for Kyber), public keys must be serialized unambiguously.

The Vulnerability: If public keys are represented as untyped byte arrays without domain separation:

$$\text{encodeEC}(pk) = \text{encodeKEM}(pk)$$

an adversary can substitute Bob's classical Signed Prekey (SPK_B) for a KEM Prekey ($PQPK_B$).

Attack Walkthrough:

1. The server delivers Bob's EC key SPK_B in place of a Kyber prekey.

2. Alice attempts to execute KEM encapsulation against an EC public key point.

3. If the mathematical parser coerces the bitstring into a degenerate lattice element, the resulting encapsulation produces an attacker-predictable or zeroed shared secret (`weakECasKEM`).

4. The attacker can now strip the post-quantum protection entirely, degrading the hybrid protocol into pure classical X3DH.

---

4. Bounded Reproduction & Formal Verification (Activity 3)

We implemented and verified the formal models using **ProVerif 2.05**, a symbolic protocol verifier based on the applied pi-calculus.

```
+------------------------------------------------------------------------------------+
|                        ProVerif 2.05 Automated Verification Suite                  |
+------------------------------------------------------------------------------------+
|  Model 01: 01_pqxdh_rev1_reencaps_vulnerable.pv  ==>  ATTACK TRACE RECONSTRUCTED    |
|  Model 02: 02_pqxdh_rev2_hardened.pv             ==>  INJECTIVE AGREEMENT PROVED TRUE |
+------------------------------------------------------------------------------------+
```

4.1 Activity 3 Baseline: PQXDH Revision 1 (`01_pqxdh_rev1_reencaps_vulnerable.pv`)

In this model, Bob registers multiple prekeys (`useOPK_1` , `useOPK_2`). The Associated Data in the AEAD encryption is modeled strictly according to Signal Revision 1:

```
let ad = concat2(encodeEC(IKa), encodeEC(IKb)) in
let c = senc(m, kdf_rev1(DH1, DH2, DH3, DH4, SS), ad) in
```

We define an injective agreement query checking if Bob finishing a session with Alice implies Alice initiated that exact session with the identical prekey:

```
query a: G, b: G, useOPK: bitstring, opk: G, spk: G, pqpk: bitstring, ts: bitstring;
  inj-event(RespondDone(b, a, useOPK, opk, spk, pqpk, ts)) ==>
  inj-event(InitDone(a, b, useOPK, opk, spk, pqpk, ts)).
```

Automated ProVerif Execution Result:

` text

Starting query inj-event(RespondDone(...)) ==> inj-event(InitDone(...))

goal reachable: event(RespondDone(b[],a[],useOPK_2,opk,spk,pqpk_1,ts))

...

RESULT event(RespondDone(b,a,useOPK_2,opk,spk,pqpk,ts))@i_1 ==>

(useOPK_2 = True && event(InitDone(a,b,True,opk,spk,pqpk,ts))@i) is false.

`

Attack Trace Analysis: ProVerif successfully found an attack in 23.77 seconds. In the generated execution trace:

1. Alice executes `InitDone` with parameter `pqpk = pqpk_2` (the attacker-compromised key).

2. The Dolev-Yao attacker intercepts the message, takes $c = \text{senc}(m, \dots)$, decapsulates ct_{pq} to recover ss, and re-encapsulates ss under `pqpk_1` using `reencaps(ct, sk2, pqpk1)` .

3. Bob receives the re-encapsulated ciphertext, decapsulates it with `sk1` , derives the identical key, and fires `RespondDone` with `pqpk = pqpk_1` .

4. The event arguments mismatch (`pqpk_1` vs `pqpk_2`), falsifying the injective agreement query.

4.2 Activity 3 Corrected: PQXDH Revision 2 (`02_pqxdh_rev2_hardened.pv`)

In Revision 2, the KEM public key is explicitly bound into the Associated Data:

```
let ad = concat3(encodeEC(IKa), encodeEC(IKb), pqpk) in
let c = senc(m, kdf_rev2(DH1, DH2, DH3, DH4, SS), ad) in
```

Automated ProVerif Execution Result:

`text

Starting query inj-event(RespondDone(...)) ==> inj-event(InitDone(...))

...

RESULT event(RespondDone(b,a,useOPK_2,opk,spk,pqpk,ts))@i_1 ==>

(useOPK_2 = True && event(InitDone(a,b,True,opk,spk,pqpk,ts))@i) is true.

`

Outcome: ProVerif proves the query in 20.73 seconds. If the attacker re-encapsulates SS under `pqpk_1`, Bob reconstructs $\text{AD} = (IK_A \parallel IK_B \parallel pqpk_1)$. Because Alice computed the AEAD ciphertext with $\text{AD} = (IK_A \parallel IK_B \parallel pqpk_2)$, Bob's AEAD tag verification fails. The attack is thwarted.

---

5. Novel Cryptographic Explorations (Activity 4)

```
+-------------------------------------------------------------------------------------+
|                  Activity 4 Engineering Explorations in ProVerif 2.05               |
+-------------------------------------------------------------------------------------+
|  Exploration 1: 03_activity4_confuse_kem_ec.pv   ==>  ATTACK PROVED / UNPROVABLE     |
|  Exploration 2: 04_activity4_pqxdh_kdf_bound.pv  ==>  INJECTIVE AGREEMENT PROVED TRUE |
+-------------------------------------------------------------------------------------+
```

5.1 Exploration 1: Algorithm and Key-Type Confusion (`03_activity4_confuse_kem_ec.pv`)

In standard Signal specifications, public keys are formatted with type tags. We asked: *What occurs if an implementation omits key prefixes and treats EC public keys and KEM public keys as the same type?*

Formal Model Construction: We unified the encoding equation so that an EC point can be interpreted as a KEM key:

`proverif

fun encodeEC(G): bitstring.

fun encodeKEM(kem_pk): bitstring.

(* Unification: Attacker can supply encodeEC where encodeKEM is expected *)

reduc forall x: G; asKEM(encodeEC(x)) = kem_pk_from_ec(x).

fun encaps_weak(G): bitstring.

equation forall x: G; encaps(kem_pk_from_ec(x)) = (encaps_weak(x), weakECasKEM).

`

ProVerif Query: Can an attacker force the initiator to accept a session where the post-quantum shared secret is degenerated to `weakECasKEM`?

` proverif

query a: G, b: G, useOPK: bitstring, opk: G, spk: G, pqpk: bitstring, ts: bitstring;

event(InitDone(a, b, useOPK, opk, spk, pqpk, ts)) && attacker(weakECasKEM).

`

Automated Result: ProVerif immediately confirms:

` text

RESULT event(InitDone(a,b,useOPK_2,opk,spk,pqpk,ts))@i_1 && attacker(weakECasKEM)

cannot be proved (attack trace found).

`

Cryptanalytic Takeaway: Without unambiguous domain separation in the serialization layer, an attacker can manipulate prekey bundles to force classical-only fallback without triggering algebraic parsing exceptions.

5.2 Exploration 2: Direct KDF Transcript Binding Proposal P2 (`04_activity4_pqxdh_kdf_bound.pv`)

Signal Revision 2 fixes re-encapsulation by modifying the **AEAD layer** ($\text{AD} = IK_A \parallel IK_B \parallel PQPK_B$). However, from an architectural standpoint, relying on the symmetric application layer to patch an asymmetric key-agreement binding defect violates protocol layer encapsulation.

We implemented and verified **Revision 3 Proposal P2 (Direct KDF Transcript Binding)**:

Protocol Modification: The AEAD Associated Data remains purely classical ($\text{AD} = IK_A \parallel IK_B$), but Bob's KEM public key ($PQPK_B$) is fed directly into the root key derivation function:

$$SK = \text{HKDF-Extract}(\text{KMAC256}(DH_1 \parallel DH_2 \parallel DH_3 \parallel DH_4 \parallel SS \parallel PQPK_B))$$

ProVerif Model:

` proverif

fun concatKDF(G, G, G, G, bitstring, bitstring): bitstring.

(* Responder derives SK directly binding pqpk *)

let sk_p2 = kdf(concatKDF(dh1, dh2, dh3, dh4, ss, PQSPKB_p)) in

let ad_classical = concat2(encodeEC(IKa), encodeEC(IKb)) in

let msg = sdec(c, sk_p2, ad_classical) in

`

Automated Verification Result:

` text

Starting query inj-event(RespondDone(...)) ==> inj-event(InitDone(...))

...

RESULT event(RespondDone(b,a,useOPK_2,opk,spk,pqpk,ts))@i_1 ==>

(useOPK_2 = True && event(InitDone(a,b,True,opk,spk,pqpk,ts))@i) is true.

`

Architectural Advantage: Even if the application sends an unauthenticated plaintext payload, or uses an AEAD cipher that omits Associated Data, the derived session key itself is cryptographically bound to $PQPK_B$. If the attacker attempts re-encapsulation, the two parties derive completely different session keys ($SK_1 \ne SK_2$), and decryption fails naturally.

---

6. Security Evolution Matrix (Activity 5)

We synthesize our formal results and literature study across the primary paper [R1] and supporting literature [R2, R3, R4]:

Security Dimension	Primary Paper [R1] (Signal PQXDH)	Supporting Literature [R2] (Duverger et al. CCS '25)	Supporting Literature [R3] (Abiega-L'Eglisse '26)	Supporting Literature [R4] (Olushola & Meenakshi '26)
1. Classical to PQ Key Agreement	Retrofits X3DH with hybrid ML-KEM-1024 + 4 classical X25519 DH operations.	Analyzes subversion-resilient key exchange where KEM randomness is untrusted.	Clean-slate post-quantum instant messaging without legacy Diffie–Hellman components.	Standalone session protocol integrating ML-KEM, ML-DSA-44, and AES-256-GCM.
2. Harvest-Now-Decrypt-Later (HNDL)	Relies on KEM shared secret (SS) to shield classical DH against future CRQCs.	Protects against quantum adversaries even if underlying KEM implementations are backdoored.	Evaluates forward secrecy via bidirectional post-quantum ephemeral key ratchets.	Guarantees post-quantum forward secrecy via ephemeral Kyber key generation per session.
3. Protocol & Key Binding	Identified KEM re-encapsulation;	Binds public keys into deterministic	Cryptographically binds the full	Binds client public keys and

Security Dimension	Primary Paper [R1] (Signal PQXDH)	Supporting Literature [R2] (Duverger et al. CCS '25)	Supporting Literature [R3] (Abiega-L'Eglisse '26)	Supporting Literature [R4] (Olushola & Meenakshi '26)
	resolved via AEAD Associated Data ($\text{AD} = IK_A \parallel IK_B \parallel PQPK$).	transcript hashes before cryptographic reduction.	message transcript into KDF and key confirmation tags.	ciphertexts into a cumulative transcript signed with ML-DSA.
4. Specification vs. Implementation	Spec omitted key binding; Signal client was safe only because Kyber's FO transform hashes $H(pk)$.	Demonstrates how specification gaps allow algorithm substitution during compilation.	Emphasizes strict protocol state machines over informal prose descriptions.	Highlights buffer pre-allocation and constant-time library APIs as critical implementation invariants.
5. Formal Verification Assumptions	Symbolic Dolev-Yao in ProVerif; computational reduction in CryptoVerif.	Uses game-based security proofs under algorithm-substitution attacks (ASA).	Verifies secrecy and mutual authentication using automated state-space exploration.	Evaluates empirical cryptographic timing and CPU cycle consumption.
6. Resistance to Key Confusion	Mitigated by enforcing distinct prefix bytes for X25519 vs. Kyber-1024 public keys.	Requires strict cryptographic domain separation strings across all hash inputs.	Enforces rigid ASN.1 / CBOR schema definitions for all wire-format keys.	Uses fixed-size binary structs with hardcoded protocol header identifiers.
7. Bandwidth & Network Impact	Adds ~1.5 KB per prekey bundle; acceptable for asynchronous mobile store-and-forward.	Focuses on algorithmic overhead of multi-party subversion-checking protocols.	Analyzes message bloat in mobile messaging networks and battery drain.	Demonstrates that post-quantum digital signatures (Dilithium = 2.4 KB) cause IP packet fragmentation.

7. Five Core Engineering Lessons for Post-Quantum Migration (Activity 6)

Lesson 1: The Primitive Substitution Fallacy

Insight: Replacing a classical primitive with a post-quantum counterpart is not a modular "drop-in" operation.

Analysis: Diffie–Hellman provides non-interactive commutative symmetry ($g^{ab} = g^{ba}$), allowing both parties to contribute equally to the shared secret without digital signatures. In contrast, a Key Encapsulation Mechanism is asymmetric: the responder creates a keypair, and the initiator

encapsulates. Dropping a KEM into a protocol without adjusting the authentication state machine creates active Man-in-the-Middle and key-substitution vulnerabilities. Post-quantum migration requires a full re-engineering of the protocol handshake.

Lesson 2: Mandatory Cryptographic Transcript and Public-Key Binding

Insight: Generic IND-CCA2 security is insufficient for authenticated key exchange.

Analysis: As demonstrated in Activity 3, IND-CCA2 security only protects confidentiality under a single fixed key. To guarantee session independence and prevent re-encapsulation or splicing attacks, every public key ($IK, SPK, PQPK$), ciphertext, and nonce must be explicitly bound into the key derivation context:

$$SK = \text{HKDF}(\text{Transcript} = \text{Hash}(pk_A \parallel pk_B \parallel ct \parallel \text{nonces}))$$

Our Proposal P2 verified in Activity 4 proves that direct KDF binding is superior to patching bindings at the symmetric AEAD layer.

Lesson 3: Bandwidth, MTU, and Asynchronous Store-and-Forward Constraints

Insight: Post-quantum public keys and signatures impose severe network-layer trade-offs.

Analysis: While Curve25519 keys are 32 bytes, ML-KEM-1024 keys are 1,568 bytes, and ML-DSA-44 signatures are 2,420 bytes. In asynchronous messaging (Signal), prekey bundles grow from under 200 bytes to over 2 KB, increasing server storage and mobile radio sleep-wake cycles. In interactive datagram protocols (UDP/VPNs), handshakes exceeding the 1,500-byte Ethernet MTU cause IP fragmentation, leading to catastrophic packet drop rates on cellular networks. Post-quantum protocols must be co-designed with transport-layer MTU constraints.

Lesson 4: Formal Verification as a Non-Negotiable Specification Gate

Insight: Functional unit tests cannot detect protocol composition vulnerabilities.

Analysis: Traditional software test suites (`assert(decaps(encaps(pk)) == ss)`) verify only that honest parties complete the protocol under ideal conditions. They cannot explore the combinatorial interleaving of messages by an active Dolev-Yao attacker. Automated formal verifiers like ProVerif must be integrated into the continuous integration (CI) pipeline of protocol specifications to mathematically prove injective agreement before deployment.

Lesson 5: Specification Independence from Implementation Artifacts

Insight: Cryptographic protocols must never rely on accidental implementation side-effects.

Analysis: In Signal PQXDH Revision 1, the KEM re-encapsulation flaw was not exploitable in production only because the chosen Kyber implementation happened to include the public key hash inside its Fujisaki–Okamoto (FO) transform: $SS = \text{KDF}(m \parallel H(pk))$. Had Signal substituted Kyber for another IND-CCA2 KEM (such as Classic McEliece or FrodoKEM without this specific transform), the protocol would have been immediately vulnerable. A protocol specification

must be self-contained and normative, guaranteeing security regardless of the underlying library implementation.

8. Difficulties, Limitations, and Deviations

Scope Bounding: Our formal verification focused on the initial hybrid key agreement handshake of PQXDH, rather than the entire multi-year Double Ratchet session lifecycle. This was an intentional design choice, as the primary post-quantum transition challenges reside in the asynchronous prekey agreement phase.

Tooling Constraints: ProVerif 2.05 operates in the symbolic model (perfect cryptography), where bitstrings cannot be partially manipulated unless explicit rewrite rules are defined. To model the KEM re-encapsulation flaw, we defined an explicit algebraic rewrite relation (`fun reencaps/3`) capturing the adversary's capability to transplant shared secrets under generic IND-CCA2 assumptions.

9. Individual Contribution Split & Generative AI Usage

9.1 Equal 3-Person Contribution Split

The project workload was divided equally into three balanced technical tracks:

```
+-------------------------------------------------------------------------------------------
---------------+
|                                    Equal 3-Person Contribution Split
|
+-------------------------------------------------------------------------------------------
---------------+
| Member 1: Akhilesh K.R (CS26E001)
|
| • Activities 1 & 2: Theoretical Foundations & Security Failure Analysis
|
| • Rigorous mathematical formulation of PQXDH hybrid X3DH + ML-KEM-1024 architecture.
|
| • Detailed cryptanalytic formulation of KEM Re-encapsulation and Key-Type Confusion
failure modes.      |
| • Activity 4 Exploration 2: Direct KDF Transcript Binding Architecture (Proposal P2)
modeling & proof.  |
| • Authoring of Technical Report Sections 1, 2, 3, and 5.2.
|
+-------------------------------------------------------------------------------------------
---------------+
| Member 2: Mysha Ali (CS26E008)
|
| • Activity 3: Bounded Formal Reproduction in ProVerif
|
```

```
| • Formal translation and verification of Signal PQXDH Revision 1 (re-encapsulation attack
extraction).   |
| • Formal verification of Signal PQXDH Revision 2 (proving injective agreement via AD
binding).          |
| • Execution, log generation, and authoring of Technical Report Section 4.
|
+-------------------------------------------------------------------------------------------
---------------+
| Member 3: D R Santhosh (CS26E004)
|
| • Activity 4 Exploration 1: Algorithm/Key-Type Confusion Exploration in ProVerif
|
| • Designed, formalized, and verified the Key-Type Confusion model (encodeEC=encodeKEM).
|
| • Reconstructed the weak key derivation failure trace (weakECasKEM).
|
| • Execution, log generation, and authoring of Technical Report Section 5.1.
|
+-------------------------------------------------------------------------------------------
---------------+
| Collaborative & Shared (All Members):
|
| • Activities 5 & 6: Synthesis of the Security Evolution Matrix across Literature [R1]–
[R4].             |
| • Distillation of the Five Core Engineering Lessons for Post-Quantum Migration.
|
| • Authoring of Presentation Slides, Video Demonstration Script, and Viva Defense
preparation.          |
+-------------------------------------------------------------------------------------------
---------------+
```

9.2 Generative AI Usage Declaration

Generative AI was employed responsibly as an assistive productivity tool for:

1. Drafting initial LaTeX and Markdown table layouts for the report and matrix;

2. Assisting in ProVerif syntax validation and formatting;

3. Structuring presentation slide bullet points to fit the 4-slide ceiling.

All theoretical deductions, applied pi-calculus models, ProVerif verification executions, attack trace analyses, and engineering conclusions were independently verified and authored by the team members.

---

10. References

1. K. Bhargavan, C. Jacomme, F. Kiefer, and R. Schmidt, "Formal Verification of the PQXDH Post-Quantum Key Agreement Protocol for End-to-End Secure Messaging," in *33rd USENIX Security

Symposium (USENIX Security 24)*, Philadelphia, PA, Aug. 2024, pp. 469–486.

2. K. Duverger, B. Gérard, and E. Prouff, "Subversion-resilient Key-exchange in the Post-quantum World," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, 2025.

3. A. F. De Abiega-L'Eglisse, J. M. Sierra, and J. C. Hernandez-Castro, "A Post-Quantum End-to-End Secure Protocol for Instant Messaging Applications," *Cryptography*, vol. 10, no. 3, p. 45, 2026.

4. A. Olushola and S. P. Meenakshi, "Design and implementation of an authenticated post-quantum session protocol using ML-KEM, ML-DSA, and AES-256-GCM," *Frontiers in Physics*, vol. 13, p. 1498112, 2026.

5. Signal Messenger, "The PQXDH Key Agreement Protocol," Revision 2, Signal Technology Foundation, Tech. Rep., 2023.